Paper Review: YOLOv10 - Advancing Real-Time End-to-End Object Detection

Aryam Ezra Mersha - FTP0220/14
Addis Ababa Science and Technology University
Submitted to: Dr. Ashenafi G.

***Abstract*—Object detection has seen remarkable advancements with the start of the "You Only Look Once" (YOLO) family of models. These models are well-known for their effective balance between speed and accuracy, which is crucial for real-time applications. This review delves into YOLOv10, which introduces significant architectural innovations and training strategies aimed at true end-to-end detection by eliminating the need for Non-Maximum Suppression (NMS) in post-processing. Its core principles, architectural design, and novel components like consistent dual assignments and holistic efficiency-accuracy-driven design are explored, providing a comparative analysis against its prominent predecessors, YOLOv8 and YOLOv9.**

***Index Terms*—Object Detection, YOLO, YOLOv10, Real-Time, End-to-End Detection, NMS-Free, Computer Vision, Deep Learning.**

I. Introduction to Object Detection and the YOLO Paradigm

Object detection is a fundamental computer vision task that involves identifying and localizing multiple objects within an image or video frame by providing bounding boxes and class labels for each detected instance [1]. Its applications are widespread, ranging from autonomous driving and video surveillance to industrial robotics and smart healthcare [1].

Historically, object detection methods evolved from complex two-stage approaches like R-CNN [1], which first proposed regions of interest and then classified them. While accurate, these methods often struggled with real-time performance. A significant shift occurred with the introduction of one-stage detectors. Among these, the YOLO series, first proposed by Joseph Redmon in 2015 [1], revolutionized the field. YOLO, an acronym for "You Only Look Once," reframed object detection as a single regression problem, directly predicting bounding box coordinates and class probabilities from full images in one evaluation [1]. This architecture divides the input image into a grid, with each grid cell responsible for predicting bounding boxes and class probabilities for objects centered within it. This approach dramatically improved inference speed, making real-time object detection more feasible while striving to maintain high accuracy, thereby capturing a "desirable mix between speed and accuracy." Over the past decade, the YOLO series has continually evolved, with each version introducing improvements to enhance this balance [1].

II. YOLOv10: Pushing the Boundaries of Real-Time End-to-End Detection

YOLOv10, introduced by Wang et al. [2] and originating from Tsinghua University, addresses key limitations in previous YOLO models, particularly the reliance on Non-Maximum Suppression (NMS) for post-processing and suboptimal model component design [2].

A. Key Innovations

1) NMS-Free Training with Consistent Dual Assignments: A major bottleneck in the deployment and latency of previous YOLO models has been the NMS step. NMS is computationally intensive and introduces hyperparameters that can affect performance [2]. YOLOv10 tackles this by introducing a novel training strategy enabling NMS-free inference. This is achieved through consistent dual assignments. During training, the model employs two heads:

1) A one-to-many assignment strategy (similar to traditional YOLOs using TaskAlignedLoss - TAL) provides rich supervisory signals, facilitating robust feature learning [1], [2].
2) A one-to-one assignment strategy ensures that each ground-truth object corresponds to a single positive prediction. This head is crucial for NMS-free operation [2].

During inference, only the one-to-one matched head is utilized, thus eliminating the NMS post-processing step entirely and reducing inference latency [2]. To ensure these two heads work harmoniously, YOLOv10 introduces a consistent matching metric, aligning the optimization goals of both branches and improving supervision quality [2].

2) Holistic Efficiency-Accuracy Driven Model Design: YOLOv10 meticulously re-evaluates and optimizes various architectural components from both efficiency and accuracy perspectives [2].

Efficiency-Driven Components:

- *Lightweight Classification Head:* Recognizing that the regression head often bears more significance for performance, YOLOv10 adopts a simpler, lightweight architecture for the classification head (e.g., two depthwise separable convolutions followed by a 1x1 convolution), reducing computational overhead without significant performance degradation [2].
- *Spatial-Channel Decoupled Downsampling (SCDown):* Traditional downsampling in YOLOs (e.g., 3x3 convolution with stride 2) simultaneously reduces spatial dimensions and transforms channel dimensions, which can be computationally expensive. YOLOv10 decouples these

operations: pointwise convolution is first used to modulate channel dimensions, followed by depthwise convolution for spatial reduction. This approach maximizes information retention while reducing computational cost and parameters [2].
- *Rank-Guided Block Design:* YOLOv10 analyzes the intrinsic rank of different stages in the network to identify redundancy. Stages with lower intrinsic rank (implying greater redundancy, typically deeper stages or larger models) utilize a Compact Inverted Block (CIB) structure. CIB employs depthwise convolutions for spatial mixing and cost-effective pointwise convolutions for channel mixing, adaptively reducing complexity where needed [2].

Accuracy-Driven Components:

- *Large-Kernel Convolution:* For smaller YOLOv10 model variants (e.g., N/S), large-kernel (e.g., 7x7) depthwise convolutions are integrated into the CIBs in deeper stages. This enlarges the receptive field and enhances model capability with minimal I/O overhead, often coupled with structural reparameterization to alleviate optimization issues [2].
- *Partial Self-Attention (PSA) Module:* To incorporate global modeling capabilities efficiently, YOLOv10 introduces a PSA module. Features are partitioned, with only one part fed into a multi-head self-attention (MHSA) and feed-forward network (FFN) block. This significantly reduces the computational complexity and memory footprint typically associated with self-attention, while still enhancing model capability. PSA is typically placed after Stage 4 (lowest resolution) [2].

III. YOLOv10 Architecture in Detail

The YOLOv10 architecture, while building on the foundational principles of its predecessors (YOLOv8 being a key baseline [2]), incorporates the aforementioned novel components to achieve its end-to-end and efficiency-accuracy goals. The general structure consists of a backbone, neck, and head.

A. Backbone

The backbone is responsible for extracting hierarchical features from the input image.

- *Stem:* Initial convolutional layers perform early downsampling (e.g., Conv blocks with stride 2).
- *Main Stages:* Composed primarily of C2f blocks (similar to YOLOv8) and, critically, C2fCIB blocks. The C2fCIB block is a modified C2f where the internal Bottleneck blocks are replaced by the more efficient CIBs, especially in stages identified as redundant by the rank-guided design [3].
- *Downsampling Layers:* Utilizes the Spatial-Channel Decoupled Downsampling (SCDown) blocks instead of standard strided convolutions between some stages for improved efficiency.

B. Neck

The neck aggregates features from different stages of the backbone to provide richer multi-scale representations.

- It typically employs a Path Aggregation Network (PAN) like structure, combining features through both top-down and bottom-up pathways [1].
- *SPPF (Spatial Pyramid Pooling Fast):* Positioned after the last backbone stage, SPPF captures context at multiple scales.
- *PSA (Partial Self-Attention):* Integrated into the neck, usually at the lowest resolution feature map (e.g., after SPPF or P5 stage) to capture global information efficiently.
- Upsampling and concatenation operations are used to fuse features from different levels. Blocks like C2fCIB or C2f are also used within the neck to refine fused features.

C. Head (v10Detect)

The head is responsible for making the final detections (bounding boxes and class probabilities).

- YOLOv10 uses multiple detection heads for objects of different scales (small, medium, and large).
- For its NMS-free operation, the crucial aspect is that during inference, the one-to-one assignment head (trained via dual assignments) is used.
- The classification part of the head is designed to be lightweight, using depthwise convolutions, distinct from the potentially more complex regression part [2]. This distinguishes it from the heads in YOLOv8/v9 which typically have similar structures for classification and regression [3].

IV. Comparative Analysis: YOLOv10 vs. YOLOv8 and YOLOv9

YOLOv10 demonstrates significant improvements over its predecessors, YOLOv8 and YOLOv9, in terms of both end-to-end latency and computational efficiency while maintaining or exceeding accuracy.

A. Performance Metrics (COCO Dataset)

- *Accuracy vs. Latency:* YOLOv10 consistently achieves better AP (Average Precision) at lower latencies compared to YOLOv8 and YOLOv9 across various model scales (N, S, M, B, L, X). For instance, YOLOv10-S is 1.8x faster than RT-DETR-R18 with a similar AP, and YOLOv10-B shows 46% less latency than YOLOv9-C for the same performance [2].
- *Parameter Efficiency:* YOLOv10 models generally have fewer parameters for comparable or better AP. YOLOv10-L outperforms YOLOv8-L by 0.3 AP with 1.8x fewer parameters, and YOLOv10-X outperforms YOLOv8-X by 0.5 AP with 2.3x fewer parameters [2]. Compared to YOLOv9-M/YOLO-MS, YOLOv10-M achieves similar AP with 23%/31% fewer parameters, respectively [2].
- *FLOPs:* YOLOv10 also shows reduced FLOPs. For example, YOLOv10-N/S/M/L/X show

28%/36%/41%/44%/57% fewer parameters and 23%/24%/25%/27%/38% less computation (FLOPs) compared to their YOLOv8 counterparts [2].
- *NMS-Free Advantage:* The most significant latency reduction comes from the NMS-free design. When comparing end-to-end latency (including NMS for v8/v9), YOLOv10's direct output is substantially faster [2]. For instance, YOLOv10-S shows an end-to-end latency of 2.49ms, while YOLOv8-S is 7.07ms [2].

B. Architectural Distinctions

1) YOLOv10 vs. YOLOv8:

- *NMS:* YOLOv10 is NMS-free; YOLOv8 requires NMS.
- *Head:* YOLOv10 uses a lightweight classification head and a dual-assignment strategy for the head; YOLOv8 uses a standard coupled head.
- *Core Blocks:* YOLOv10 introduces C2fCIB (with CIBs) and PSA; YOLOv8 primarily uses C2f with Bottleneck blocks.
- *Downsampling:* YOLOv10 uses SCDown; YOLOv8 uses standard strided convolutions [1], [3].

2) YOLOv10 vs. YOLOv9:

- *NMS:* YOLOv10 is NMS-free; YOLOv9 requires NMS.
- *Core Concepts:* YOLOv9 introduced Programmable Gradient Information (PGI) and Generalized ELAN (GELAN) [1], [3]. YOLOv10 focuses on NMS-free design, efficient block structures (CIB, PSA), and decoupled components.
- *Auxiliary Components:* YOLOv9 features an auxiliary head for training [3]; YOLOv10 uses a dual-head (one-to-many/one-to-one) strategy specifically for NMS-free training [1].

C. Qualitative Differences

- *Small Objects:* Some practical comparisons suggest YOLOv8 and YOLOv9 might perform better on very small objects than pre-fine-tuned YOLOv10, potentially due to YOLOv10's NMS-free design requiring highly discriminative features or specific tuning for such cases. However, YOLOv10's design includes large-kernel convolutions for smaller models precisely to enhance capability, including small object detection.
- *Low-Light/Many Objects:* YOLOv10 is noted to struggle in some low-light conditions and with scenes containing many objects in some qualitative tests, more so than YOLOv8/v9. This could indicate a need for specific data augmentation or fine-tuning for such challenging scenarios.
- *False Positives:* YOLOv10, by eliminating NMS, aims to reduce redundant boxes. Some initial observations suggested YOLOv10 had fewer false positives than YOLOv8, while other specific scenarios highlighted different types of false positives. The nature of false positives can change with NMS-free designs.
- *Stability:* YOLOv8 and v9 were perceived as "relatively more stable" in some varied conditions in video comparisons. This often comes down to the maturity of the training strategies and robustness across diverse, unconstrained data, an area where NMS-free models might require more extensive tuning.

V. Discussion

YOLOv10 presents a compelling advancement in the YOLO lineage, primarily through its successful implementation of an NMS-free architecture. This significantly reduces end-to-end inference latency, a critical factor for real-time deployment. The efficiency-accuracy-driven design, with components like the lightweight classification head, spatial-channel decoupled downsampling, rank-guided CIB blocks, and partial self-attention, demonstrates a detailed approach to optimizing computational resource utilization while enhancing model capability.

However, the NMS-free approach, especially for smaller models, can present challenges. As noted in the YOLOv10 paper itself and corroborated by some practical observations, there can be a performance gap when compared to traditional one-to-many training with NMS, particularly for smaller YOLOv10 variants on benchmarks like COCO [2]. This suggests that NMS-free models demand highly discriminative features, which smaller models with limited capacity might struggle to produce adequately. The authors of YOLOv10 acknowledge this and suggest future work to reduce this gap [2].

Furthermore, the qualitative comparisons highlight that out-of-the-box performance in highly diverse or challenging conditions (e.g., low light, extreme clutter) may still benefit from specific fine-tuning or data augmentation strategies tailored to the NMS-free paradigm. The "stability" observed in older models like YOLOv8 and YOLOv9 might stem from more mature training regimens and the inherent filtering provided by NMS.

VI. Conclusion

YOLOv10 stands as a significant milestone in the evolution of real-time object detection. By successfully integrating an NMS-free training and inference pipeline alongside a suite of carefully designed architectural optimizations, it achieves state-of-the-art performance in terms of latency-accuracy trade-offs. Its contributions, particularly the consistent dual assignments for NMS-free training and the holistic efficiency-accuracy-driven model design offer valuable insights for future research in efficient and deployable computer vision models. While challenges remain, especially in maximizing the performance of smaller NMS-free models and ensuring robustness across all conditions without extensive tuning, YOLOv10 effectively pushes the performance-efficiency boundary for the YOLO family and sets a new benchmark for end-to-end object detectors.

REFERENCES

[1] C.-Y. Wang and H.-Y. M. Liao, "YOLOv1 to YOLOv10: The fastest and most accurate real-time object detection systems," *YOLO SURVEY, AUGUST 2024*, 2024.

[2] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-Time End-to-End Object Detection," *arXiv preprint arXiv:2405.14458*, 2024. [Online]. Available: https://arxiv.org/abs/2405.14458

[3] P. Hidayatullah, N. Syakrani, M. R. Sholahuddin, T. Gelar, and R. Tubagus, "YOLOv8 to YOLO11: A Comprehensive Architecture In-depth Comparative Review," *A PREPRINT, January 23, 2025*, 2025.

[4] THU-MIG, "YOLOv10," https://github.com/THU-MIG/yolov10, May 2024.

[5] Ultralytics, "YOLOv10 Model Documentation," https://docs.ultralytics.com/models/yolov10/, 2024.